Admin

Last "normal" week

Lab 7, Assign 7 interrupts

Squash those P1 issues! Retest accepted up to Sat 5/30

Looking ahead to projects

Form team (2 is best)

Interrupts (resumed)

Last time

Exceptional control flow, low-level mechanisms

Today

Design of module to manage interrupt system

Implementation to handle details, dispatch to handlers

Public interface that is safe, convenient, flexible

Client code to configure, enable, process interrupts

Coordination of activity

Data sharing, code designed so that it can be safely interrupted

Interrupts (so far)

Top-level interrupts system configuration

- `mtvec` CSR to set trap handler address
- `mstatus`, `mie` CSRs to enable interrupts, switch on

Transfer of control to enter/exit interrupt code

- Assembly to preserve registers
- Call into C code

Wait, what code is this again?

- Assembly to restore registers, resume interrupted code

Each interrupt processed in same manner

- Jump to address in **`mtvec`** (e.g **`trap_handler`**)
- Single function responds to any/all interrupt events
- How to allow different response to timer event vs. button event vs. key event?

Platform-Level Interrupt Controller

3.8 Platform-Level Interrupt Controller (PLIC)

3.8.1 Overview

The PLIC is only used for sampling, priority arbitration and distribution for external interrupt sources.

- Sampling, priority arbitration and distribution for external interrupt sources
- The interrupt can be configured as machine mode and super user mode
- Up to 256 interrupt source sampling, supporting level interrupt and pulse interrupt
- 32 levels of interrupt priority
- Maintains independently the interrupt enable for each interrupt mode (machine/super user)
- Maintains independently the interrupt threshold for each interrupt mode (machine/super user)
- Configurable access permission for PLIC registers

3.8.3 Register List

Module Name	Base Address
RISCV PLIC	0x10000000

Register Name	Offset	Description
PLIC_PRIO_REGn	0x0000+n*0x0004 (0<n<256)	PLIC Priority Register n
PLIC_IP_ REGn	0x1000+n*0x0004 (0≤n<9)	PLIC Interrupt Pending Register n
PLIC_MIE_ REGn	0x2000+n*0x0004 (0≤n<9)	PLIC Machine Mode Interrupt Enable Register n
PLIC_SIE_ REGn	0x2080+n*0x0004 (0≤n<9)	PLIC Superuser Mode Interrupt Enable Register n
PLIC_CTRL_REG	0x1FFFFC	PLIC Control Register
PLIC_MTH_REG	0x200000	PLIC Machine Threshold Register
PLIC_MCLAIM_REG	0x200004	PLIC Machine Claim Register
PLIC_STH_REG	0x201000	PLIC Superuser Threshold Register

[D1-H User Manual Ch 3.8]

[RISC-V PLIC Specification]

Interrupt sources

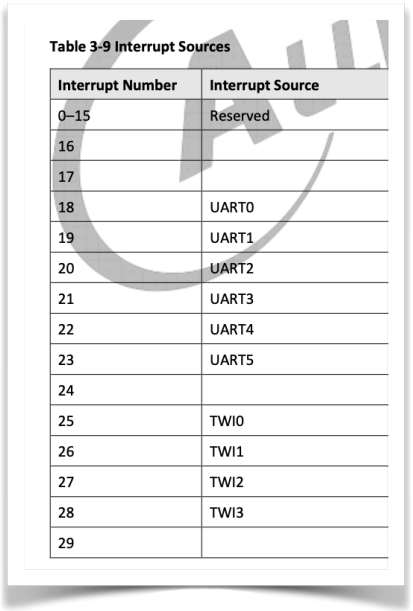
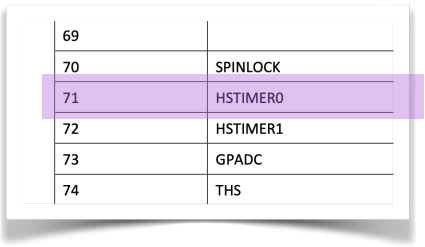
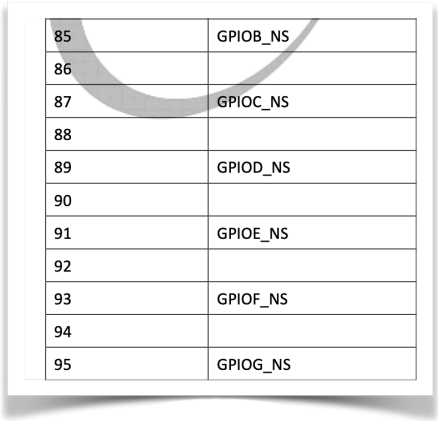


Table 3-9 Interrupt Sources

Interrupt Number	Interrupt Source
0–15	Reserved
16	
17	
18	UART0
19	UART1
20	UART2
21	UART3
22	UART4
23	UART5
24	
25	TWI0
26	TWI1
27	TWI2
28	TWI3
29	



69	
70	SPINLOCK
71	HSTIMER0
72	HSTIMER1
73	GPADC
74	THS



85	GPIOB_NS
86	
87	GPIOC_NS
88	
89	GPIOD_NS
90	
91	GPIOE_NS
92	
93	GPIOF_NS
94	
95	GPIOG_NS

```
enum interrupt_source_t {
    INTERRUPT_SOURCE_UART0 = 18,
    INTERRUPT_SOURCE_UART1 = 19,
    INTERRUPT_SOURCE_UART2 = 20,
    INTERRUPT_SOURCE_UART3 = 21,
    INTERRUPT_SOURCE_UART4 = 22,
    INTERRUPT_SOURCE_UART5 = 23,
    INTERRUPT_SOURCE_HSTIMER0 = 71,
    INTERRUPT_SOURCE_HSTIMER1 = 72,
    INTERRUPT_SOURCE_GPIOB = 85,
    INTERRUPT_SOURCE_GPIOC = 87,
    INTERRUPT_SOURCE_GPIOD = 89,
    INTERRUPT_SOURCE_GPIOE = 91,
    INTERRUPT_SOURCE_GPIOF = 93,
    INTERRUPT_SOURCE_GPIOG = 95,
};
```

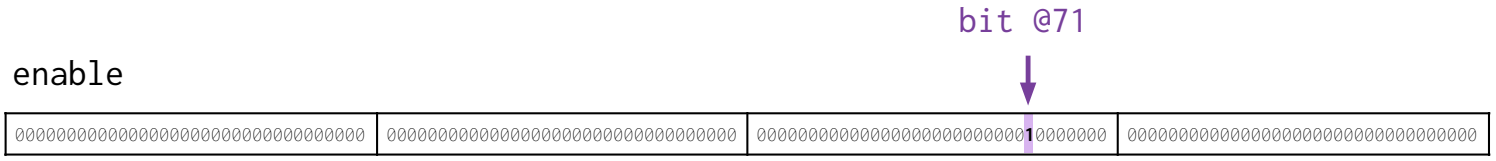
Interrupt source each assigned number

256 possible sources, 0 to 255

PLIC registers

Enable registers: one bit per interrupt source

`INTERRUPT_SOURCE_HSTIMER0 = 71`



Set bit 71 enables source #71 to generate interrupts

In `trap_handler`, read claim register to identify which source id

3.8.3 Register List

Module Name	Base Address
RISCV PLIC	0x10000000

Register Name	Offset	Description
PLIC_PRIO_REGn	0x0000+n*0x0004 (0<n<256)	PLIC Priority Register n
PLIC_IP_ REGn	0x1000+n*0x0004 (0≤n<9)	PLIC Interrupt Pending Register n
PLIC_MIE_ REGn	0x2000+n*0x0004 (0≤n<9)	PLIC Machine Mode Interrupt Enable Register n
PLIC_SIE_ REGn	0x2080+n*0x0004 (0≤n<9)	PLIC Superuser Mode Interrupt Enable Register n
PLIC_CTRL_REG	0x1FFFFC	PLIC Control Register
PLIC_MTH_REG	0x200000	PLIC Machine Threshold Register
PLIC_MCLAIM_REG	0x200004	PLIC Machine Claim Register
PLIC_STH_REG	0x201000	PLIC Superuser Threshold Register
PLIC_SCLAIM_REG	0x201004	PLIC Superuser Claim Register

Dispatch to handler

Function pointers save the day!

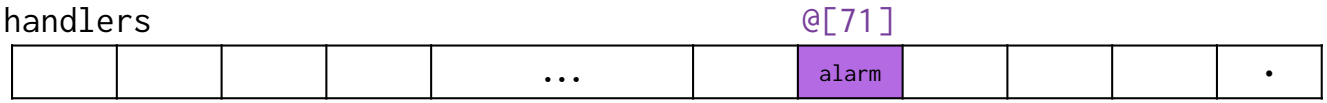
Track handler function per-source

Use source id to lookup associated handler function

Store fn pointers in array indexed by source id, fast lookup

```
void _trap_handler(void)  __attribute__((interrupt("machine"))) {
    long mcause = CSR_READ(mcause);
    if (mcause == EXTERNAL_INTERRUPT) {
       uint32_t source_id = module.plic->regs.claim_complete;
       handlers[source_id].fn();  // call handler for source_id
```

Module has array of function pointers, mirrors structure of peripheral registers



INTERRUPT_SOURCE_HSTIMER0 = 71

Set handler for source

Client sets handler (function pointer) for interrupt source

Store function pointers in array, one per interrupt source

- Interrupt source number is index into array

Top-level trap handler dispatches to per-source handler

- Claim register has interrupt source number, use as index into array

Aux data can be used to pass information into handler function

- If not needed, aux data can be `NULL`
- Data type is `void *` for flexibility

Review our code in `$CS107E/src/interrupts.c`

Design goals of `interrupts` module

Convenience, safety

- Abstract away details
- Simple consistent interface
- Defend against mis-use, avoid runtime failures (debugging!)
  - e.g. confirm source, report steps out of order, don't enable without handler

Flexible

- Support different use cases (individual handler per interrupt source, client data)

Speed

- Minimize number of cycles spent in library
  - Ideally quick and direct handoff to client function

Client use of interrupts

1. Configure/enable peripheral to generate interrupt on event

    e.g. falling edge on gpio pin, countdown timer elapsed, char received on uart

2. Write function to process event, set as handler

3. Globally enable interrupts

Interrupt generated if and only if all steps done

Forgetting one is a common bug

HSTimer

D1-H User Manual Ch 3.7

3.7 High Speed Timer

3.7.1 Overview

The high speed timer (HSTimer) module consists of HSTimer0 and HSTimer1. HSTimer0 and HSTimer1 are down counters that implement timing and counting functions. They are completely consistent. Compared with the timer module, the HSTimer module provides more precise timing and counting.

The HSTimer has the following features:

- Single clock source: AHB0
- Supports 5 prescale factors
- Configurable 56-bit down timer
- Supports 2 timing modes: periodic mode and one-shot mode
- Supports the test mode
- Generates an interrupt when the count is decreased to 0

Module Name	Base Address
HSTimer	0x03008000

Register Name	Offset	Description
HS_TMR_IRQ_EN_REG	0x0000	HS Timer IRQ Enable Register
HS_TMR_IRQ_STAS_REG	0x0004	HS Timer Status Register
HS_TMR0_CTRL_REG	0x0020	HS Timer0 Control Register
HS_TMR0_INTV_LO_REG	0x0024	HS Timer0 Interval Value Low Register
HS_TMR0_INTV_HI_REG	0x0028	HS Timer0 Interval Value High Register
HS_TMR0_CURNT_LO_REG	0x002C	HS Timer0 Current Value Low Register
HS_TMR0_CURNT_HI_REG	0x0030	HS Timer0 Current Value High Register

3.7.3.6 Processing the HSTimer Interrupt

Follow the steps below to process the HSTimer interrupt:

Step 1 Enable interrupt: Write the corresponding interrupt enable bit of HS_TMR_IRQ_EN_REG, when the counting time of HSTimer reaches, the corresponding interrupt generates.

Step 2 After entering the interrupt process, write HS_TMR_IRQ_STAS_REG to clear the interrupt pending.

Step 3 Resume the interrupt and continue to execute the interrupted process.

`hstimer` module

Config timer

- **`hstimer_init(id, interval, mode)`**

  Countdown interval in microseconds, mode is one shot or periodic

Set handler

- **`hstimer_set_handler(id, fn, aux_data)`** call fn to handle timer interrupt event (will register fn with top-level interrupts dispatcher)

Enable

- **`hstimer_enable(id)`** starts countdown, interrupt when countdown reaches zero

Clear interrupt

- **`hstimer_interrupt_clear(id)`** clears event

References

- Interface **`$CS107E/include/hstimer.h`** Implementation **`$CS107E/src/hstimer.c`**

GPIO interrupts

9.7.3.6 Interrupt

Each group IO has an independent interrupt number. The IO within-group uses one interrupt number when one IO generates interrupt, the GPIO pins sent interrupt request to the interrupt controller. External Interrupt Status Register is used to query which IO generates interrupt.

The interrupt trigger of GPIO supports the following trigger types.

- Positive Edge: When a low level changes to a high level, the interrupt will generate. No matter how long a high level keeps, the interrupt generates only once.
- Negative Edge: When a high level changes to a low level, the interrupt will generate. No matter how long a low level keeps, the interrupt generates only once.
- High Level: Just keep a high level and the interrupt will always generate.
- Low Level: Just keep a low level and the interrupt will always generate.
- Double Edge: Positive and negative edge.

External Interrupt Configure Register is used to configure the trigger type.

The GPIO interrupt supports hardware debounce function by setting External Interrupt Debounce Register. Sample trigger signal using a lower sample clock, to reach the debounce effect because the dither frequency of the signal is higher than the sample frequency.

[D1-H User Manual Ch 9.7.3.6]

PB_EINT_CFG0	0x0220	PB External Interrupt Configure Register 0
PB_EINT_CTL	0x0230	PB External Interrupt Control Register
PB_EINT_STATUS	0x0234	PB External Interrupt Status Register
PB_EINT_DEB	0x0238	PB External Interrupt Debounce Register
PC_EINT_CFG0	0x0240	PC External Interrupt Configure Register 0
PC_EINT_CTL	0x0250	PC External Interrupt Control Register
PC_EINT_STATUS	0x0254	PC External Interrupt Status Register
PC_EINT_DEB	0x0258	PC External Interrupt Debounce Register
PD_EINT_CFG0	0x0260	PD External Interrupt Configure Register 0
PD_EINT_CFG1	0x0264	PD External Interrupt Configure Register 1

Gpio events

Config event for pin

- `gpio_interrupt_config(pin, event, debounce)`

  Low/high state, rising/falling edge, debounce will coalesce

Set handler

- `gpio_interrupt_set_handler(pin, fn, aux_data)` call fn to handle gpio event

Clear interrupt

- `gpio_interrupt_clear(pin)` clears event

References

- Interface `$CS107E/include/gpio_interrupt.h`

`gpio_interrupt` module

Single interrupt source shared by all GPIO pins within a group

- Need another level of dispatch to support per-pin handler

`gpio_interrupt_init` registers handler with top-level **`interrupts`** module

- Shared gpio_interrupt handler receives all gpio events, further dispatches to client's per-pin handler

Internal structure of **`gpio_interrupt`** similar to top-level **`interrupts`**

- Array of handlers, one per pin in group
- Scan event pending register, count zero bits, stop at first set bit, this is index into handler array

Review our code in **`$CS107E/src/gpio_interrupt.c`**

Client handler function

```
typedef void (*handlerfn_t)(void *);
```

One argument: client data **`void*`**

Ordinary C function (save/resume managed by top-level trap handler)

Handler operation should be lean (fast in & out!)

Handler must clear event!

- Otherwise event will continue to re-fire interrupt

Interrupt checklist for client

Client must:

- ✅ **Initialize interrupts module (and possibly gpio_interrupt module)**

Event-specific:

- ✅ **Config interrupts for event**
  - E.g., hstimer count down from 100
- ✅ **Write handler function to process event**
  - Handler acts on event and **clears** it
- ✅ **Set handler for event**
  - Handler will be called to process event

- ✅ **Globally enable interrupts**
  - `interrupts_global_enable` (flip switch on when everything ready)

All steps essential

Fiddly code, easy to forget steps, mix up or do in wrong order

Typical symptom is absence of action, revisit checklist to find what's missing

Sample client code

```
void timer_event(void *aux_data) {
   hstimer_interrupt_clear(HSTIMER0);
   uart_putchar('T');
}

void button_click(void *aux_data) {
   gpio_interrupt_clear(BUTTON);
   uart_putchar('B');
}

void config_timer(void) {
    hstimer_init(HSTIMER0, 1000000, HSTIMER_PERIODIC);
    hstimer_enable(HSTIMER0);
    hstimer_set_handler(HSTIMER0, timer_event, NULL);
}

void config_button(void) {
    gpio_interrupt_init();
    gpio_interrupt_config(BUTTON, GPIO_INTERRUPT_NEGATIVE_EDGE, true);
    gpio_interrupt_set_handler(BUTTON, button_click, NULL);
}

void main(void)  {
    uart_init();
    interrupts_init();
    config_timer();
    config_button();
    interrupts_global_enable();
    while (1) ;
}
```

code/interrupt_party

What's left?

An interrupt can fire at any time

- Interrupt handler adds a PS/2 scancode to a queue
- What if interrupts `main` right as it is removing scancode from same queue?
- Need to maintain integrity of shared queue

Must write code so that it can be safely interrupted

Atomicity

Global variable

mainline

```
static int g_count;
```

interrupt handler

```
g_count--;
```

```
g_count++;
```

Q. Can an update to **`g_count`** *be lost when switching between these two code paths?*

Q. What is the atomic (i.e., indivisible) unit of computation?

`Demo: code/race`

A problem

mainline

```
g_count--;
```

```
la      a4,g_count
lw      a5,0(a4)
addiw   a5,a5,-1
sw      a5,0(a4)
```

```
static int g_count;
```

interrupt handler

```
g_count++;
```

```
la      a4,g_count
lw      a5,0(a4)
addiw   a5,a5,1
sw      a5,0(a4)
```

What if interrupt fires between these two instructions?

Resume of mainline uses value previously loaded into **a5**.
What happened to increment done by interrupt handler?

Disabling interrupts

mainline

```
interrrupts_global_disable();
g_count--;
interrupts_global_enable();
```

interrupt handler

```
g_count++;
```

Temporarily disable interrupts to create "critical section"

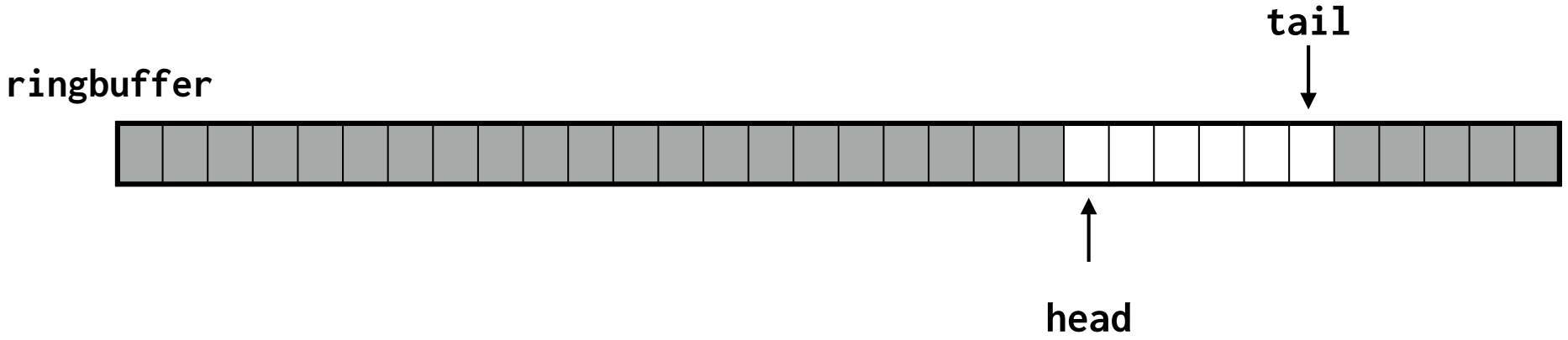
Q. Why does interrupt handler not need similar bracketing?

Safe ringbuffer

A simple approach to avoid interference is for different code paths to not write to same variables

Queue implemented as ring buffer:

- Enqueue (interrupt) writes element to tail, advances tail
- Dequeue (main) reads element from head, advances head



Ringbuffer code

```
void rb_enqueue(rb_t *rb, int elem)  {
     assert (!rb_full(rb));

     rb->entries[rb->tail] = elem;
     rb->tail = (rb->tail + 1) % CAPACITY; // only changes tail
}
```

```
 int rb_dequeue(rb_t *rb)  {
     assert (!rb_empty(rb));

     int front = rb->entries[rb->head];
     rb->head = (rb->head + 1) % CAPACITY; // only changes head
     return front;
 }
```

Review our code in **$CS107E/src/ringbuffer.c**

Preemption and safety

Difficult bugs. You'll learn more in CS111, CS140

Two approaches for now

1. Use simple, safe data structures
   - single writer (not always possible)
2. Otherwise, temporarily disable interrupts
   - works if used correctly, easy to get wrong

Interrupts in summary

Interrupts allow external events to preempt what's executing and run code immediately

- Needed for responsiveness, e.g., not miss PS/2 scancode during drawing
- Much activity is interrupt-driven: respond to keystrokes, network packets, disk reads, timers, etc.

Correct use of interrupts is challenging!

- Deals with many of the hardest issues in systems, tough bugs
- Design of module can help client use correctly

Assign7: update ps2 driver to read by interrupt